

ml-lab-ass-2

July 18, 2025

1 Assignment 2

1.1 Step 1: Import Libraries

```
[2]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, \
    accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt
```

1.2 Step 2: Load the Dataset

```
[3]: df = pd.read_csv("email_dataset.csv") # Replace with the actual file name
df.head()
```

```
[3]: Email No.  the  to  ect  and  for  of   a  you  hou  ...  connevey  jay  \
0   Email 1    0  0   1   0   0  0   2   0   0  ...      0   0
1   Email 2    8 13  24   6   6  2 102   1  27  ...      0   0
2   Email 3    0  0   1   0   0  0   8   0   0  ...      0   0
3   Email 4    0  5  22   0   5  1  51   2  10  ...      0   0
4   Email 5    7  6  17   1   5  2  57   0   9  ...      0   0

      valued  lay  infrastructure  military  allowing  ff  dry  Prediction
0          0   0                0          0          0  0   0            0
1          0   0                0          0          0  1   0            0
2          0   0                0          0          0  0   0            0
3          0   0                0          0          0  0   0            0
4          0   0                0          0          0  1   0            0
```

[5 rows x 3002 columns]

1.3 Step 3: Data Preprocessing

1.3.1 3.1 Remove unnecessary columns

```
[4]: # Remove email ID/name column (first column)
df.drop(df.columns[0], axis=1, inplace=True)
```

1.3.2 3.2 Split features and target

```
[5]: X = df.iloc[:, :-1] # All word count columns
y = df.iloc[:, -1] # The last column (0 = not spam, 1 = spam)
```

1.3.3 3.3 Train-test split

```
[6]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=42)
```

1.3.4 3.4 Feature scaling

```
[7]: scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

1.4 Step 4: K-Nearest Neighbors (KNN) Classifier

```
[8]: knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

print("KNN Classification Report:")
print(classification_report(y_test, y_pred_knn))
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
```

KNN Classification Report:

	precision	recall	f1-score	support
0	0.98	0.80	0.88	739
1	0.66	0.95	0.78	296
accuracy			0.85	1035
macro avg	0.82	0.88	0.83	1035
weighted avg	0.89	0.85	0.85	1035

KNN Accuracy: 0.8454106280193237

1.5 Step 5: Support Vector Machine (SVM) Classifier

```
[9]: svm = SVC(kernel='linear')
      svm.fit(X_train, y_train)

      y_pred_svm = svm.predict(X_test)

      print("SVM Classification Report:")
      print(classification_report(y_test, y_pred_svm))
      print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
```

SVM Classification Report:

	precision	recall	f1-score	support
0	0.97	0.96	0.96	739
1	0.90	0.92	0.91	296
accuracy			0.95	1035
macro avg	0.93	0.94	0.94	1035
weighted avg	0.95	0.95	0.95	1035

SVM Accuracy: 0.9468599033816425

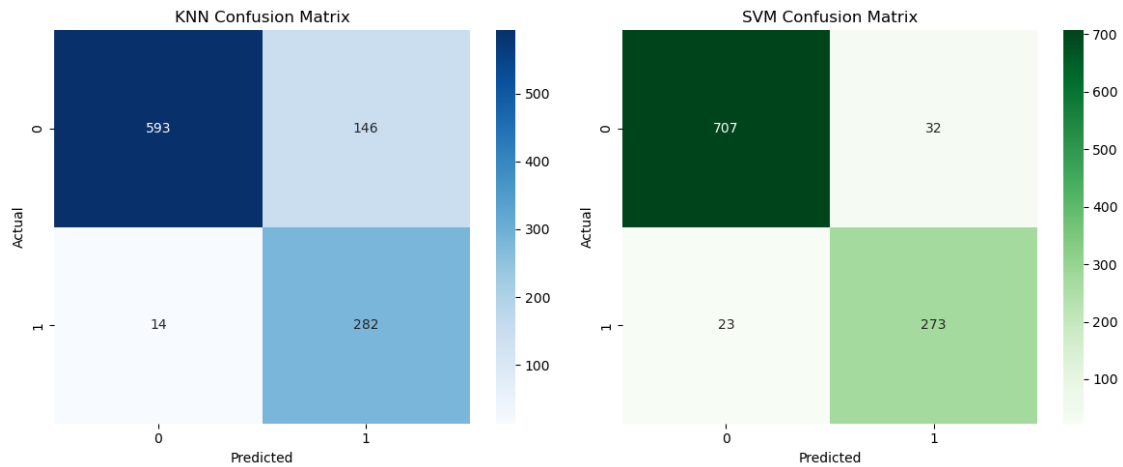
1.6 Step 6: Confusion Matrix Visualization

```
[10]: plt.figure(figsize=(12,5))

      # KNN
      plt.subplot(1,2,1)
      sns.heatmap(confusion_matrix(y_test, y_pred_knn), annot=True, fmt='d',
                  cmap='Blues')
      plt.title("KNN Confusion Matrix")
      plt.xlabel("Predicted")
      plt.ylabel("Actual")

      # SVM
      plt.subplot(1,2,2)
      sns.heatmap(confusion_matrix(y_test, y_pred_svm), annot=True, fmt='d',
                  cmap='Greens')
      plt.title("SVM Confusion Matrix")
      plt.xlabel("Predicted")
      plt.ylabel("Actual")

      plt.tight_layout()
      plt.show()
```



1.7 Observations

1. SVM performs significantly better overall, especially in reducing false positives.
2. KNN misclassifies more non-spam emails as spam — which can be annoying in practice.
3. KNN has fewer false negatives, meaning it catches slightly more spam — but at the cost of flagging too many legit emails. ## Conclusions
4. SVM is the better model for this binary classification task (spam detection).
5. It provides a higher overall accuracy, fewer misclassifications, and is more balanced in handling both classes.